



MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Laboratory work 5: Greedy algorithms

of the "Algorithm analysis" course

Checked:

asist. univ. Cristofor Fistic

Department of Software and Automation Engineering,

FCIM Faculty, UTM

Student:

Rusnac Nichita, FAF-242

Chisinau – 2026

Table of Contents

ALGORITHM ANALYSIS	3
Objective.....	3
Tasks.....	3
Theoretical Notes.....	3
MST context	3
Algorithms	3
Introduction	5
Comparison Metric	5
Input Format	6
IMPLEMENTATION	6
Prim algorithm.....	7
Kruskal algorithm.....	8
Graph generation and benchmarking setup	9
RESULTS.....	9
1. Experiment Scale and Correctness	9
2. Sparse vs Dense Behavior	10
3. Influence of Node Count	10
4. Graph Family Comparison	10
5. Practical Rule Based on (n, m)	11
CONCLUSION	12

ALGORITHM ANALYSIS

Objective

Study and analyze the greedy algorithm design technique on the **Minimum Spanning Tree (MST)** problem by implementing and comparing **Prim** and **Kruskal** algorithms on weighted undirected graphs.

The work combines:

- a theoretical overview of both algorithms (idea, complexity, memory use, applicability);
- an empirical benchmark on sparse and dense graphs;
- a scalability study with increasing graph size;
- practical recommendations for algorithm selection based on graph density and implementation details.
-

Tasks

1. Study the greedy algorithm design technique.
2. Implement Prim and Kruskal algorithms in Python.
3. Perform empirical analysis of Prim and Kruskal.
4. Increase the number of nodes in the graph and analyze how this influences runtime; present data graphically.
5. Produce a concise report with recommendations.

Theoretical Notes

A **greedy algorithm** builds a solution step-by-step by choosing the best available local option at each step. For MST, greedy choice is justified by the **cut property**: for any cut in the graph, the minimum-weight edge crossing that cut is safe to include in some MST.

MST context

Given a weighted undirected graph $G=(V,E)$, an MST is a subset of edges connecting all vertices with minimal total weight. If the graph is disconnected, the result is a **minimum spanning forest**.

Algorithms

- **Prim** grows a tree from a start vertex by repeatedly adding the lightest edge from the current tree to a new vertex (local best on the tree frontier).
- **Kruskal** builds a forest by sorting all edges by weight and adding the next lightest edge that does not create a cycle (global best among all remaining edges).

Typical implementations:

- Prim (adjacency list + binary heap):

$O(m \log n)$

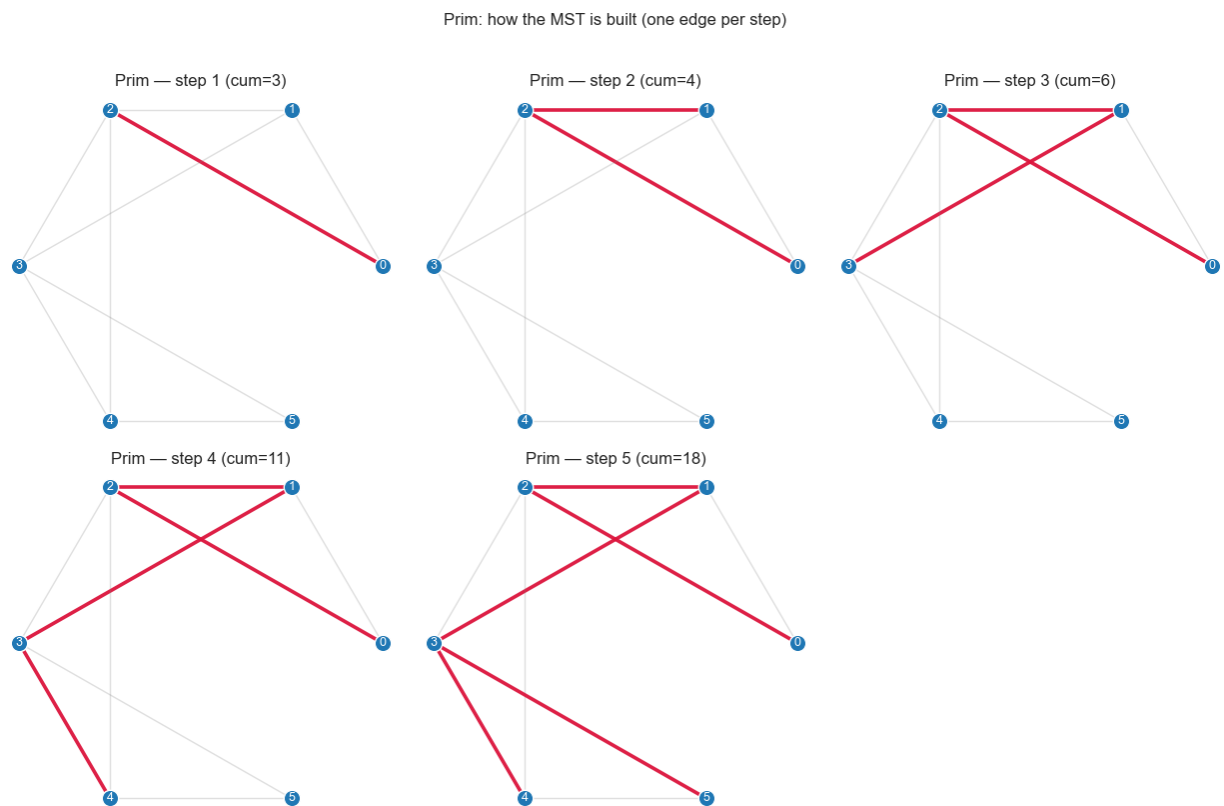


Figure 1. Prim algorithm example

- Kruskal (sort edges + DSU / Union-Find):

$O(m \log m) + O(m \alpha(n)) \approx O(m \log m)$

So both are close asymptotically. In practice (especially in Python), constants and the exact heap usage pattern can dominate.

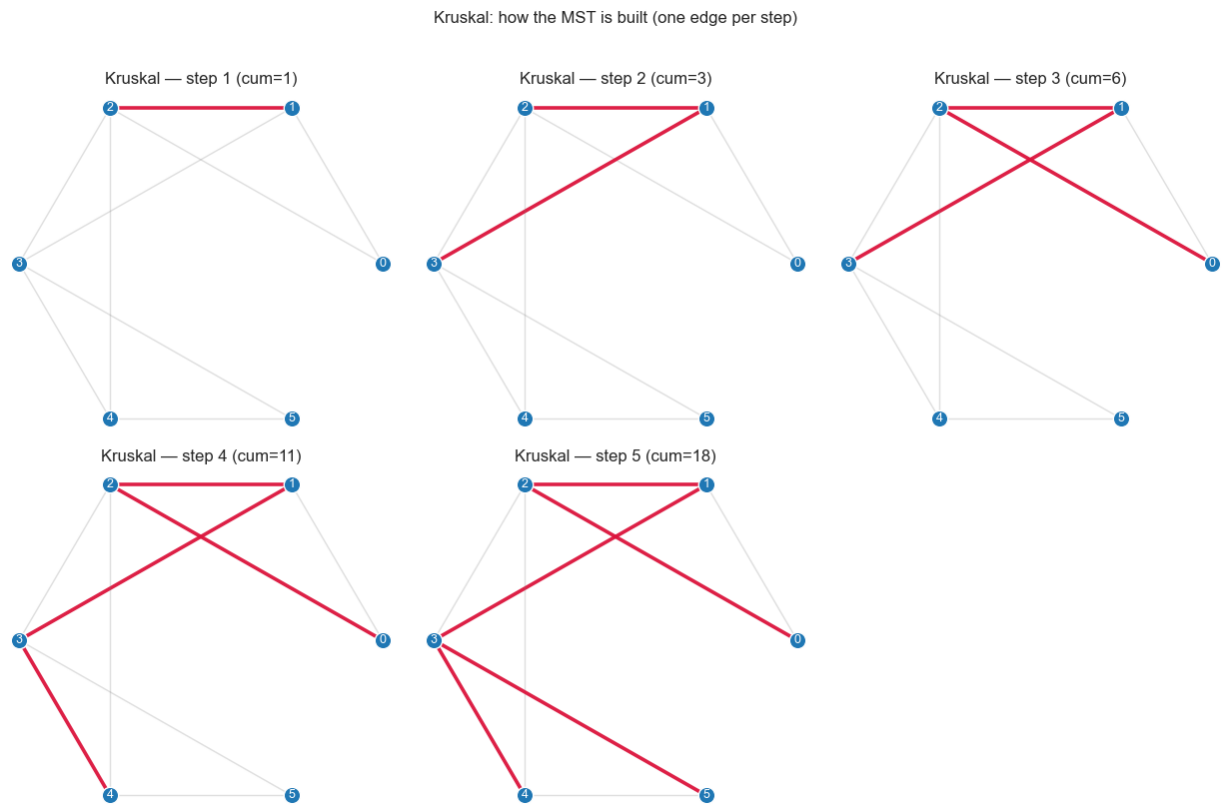


Figure 1. Prim algorithm example

Key distinction:

- Prim cost is mostly from heap operations while exploring adjacency lists.
- Kruskal cost is mostly from sorting the edge list once.

Introduction

Minimum Spanning Tree computation is fundamental in networking (minimum wiring), clustering, image segmentation, and approximation algorithms.

This laboratory compares:

- Prim (tree-growing greedy strategy)
- Kruskal (edge-sorting greedy strategy)

Focus: **practical behavior under varying density and size**, and how implementation choices affect the winner.

Comparison Metric

Primary metric: Execution time $T(n, m)$

Additional:

- correctness (Prim and Kruskal must produce the same total MST weight)
- winner frequency by density
- implementation sensitivity (baseline Prim vs optimized Prim variant)

Input Format

Graphs:

- weighted, undirected
- positive integer weights

Graph families:

- simple (connected random)
- complete
- wheel
- bipartite
- regular
- grid
- tree
- cycle
- weighted (random structure with wider weight range)

Scalability:

- nodes: $20 \rightarrow 360$
- target densities: $0.05 \rightarrow 0.90$

Density formula (undirected):

```
max_edges = n * (n - 1) / 2
density = m / max_edges
```

Note: for small n , the generator starts from a spanning tree to guarantee connectivity, so the achieved density can be larger than the requested density when m is forced to be at least $n-1$.

IMPLEMENTATION

All algorithms implemented in Python (Jupyter Notebook).

Protocol:

1. Generate graph
2. Build representations (adjacency list + edge list)
3. Run algorithms
4. Measure time (multiple repeats, averaged)
5. Validate correctness (same MST total weight)
6. Store and plot results

Prim algorithm

Complexity (typical heap-based approach):

$O(m \log n)$

Steps (conceptual):

1. Choose a start vertex in a component
2. Push all outgoing edges into a min-heap
3. Repeatedly extract the smallest edge that reaches an unvisited vertex
4. Add it to the MST and expand the frontier

Advantages:

- Works well with adjacency lists
- Natural when you already have graph adjacency

Limitations:

- A naïve frontier-edge heap can push many edges multiple times
- Constants in Python can be significant on dense graphs

Implementation:

```
def prim_mst(adj):
    n = len(adj)
    visited = [False] * n
    total = 0
    mst_edges = []
    accepted_weights = []

    for start in range(n):
        if visited[start]:
            continue

        visited[start] = True
        heap = []
        for v, w in adj[start]:
            heapq.heappush(heap, (w, start, v))

        while heap:
            w, u, v = heapq.heappop(heap)
            if visited[v]:
                continue

            visited[v] = True
            total += w
            mst_edges.append((u, v, w))
            accepted_weights.append(w)
```

```

        for to, wt in adj[v]:
            if not visited[to]:
                heapq.heappush(heap, (wt, v, to))

    return total, mst_edges, accepted_weights

```

Figure 3 – Prim algorithm

Kruskal algorithm

Steps (conceptual):

1. Sort all edges by weight
2. Scan edges from smallest to largest
3. Add an edge if it connects two different DSU components

Advantages:

- Simple structure: “sort + DSU”
- Very competitive in Python because sorting is highly optimized

Limitations:

- Requires storing all edges explicitly

Implementation:

```

class DisjointSetUnion:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra = self.find(a)
        rb = self.find(b)
        if ra == rb:
            return False

        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra

        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1

```



```

        return True

def kruskal_mst(n, edges):
    dsu = DisjointSetUnion(n)
    total = 0
    mst_edges = []
    accepted_weights = []

    for u, v, w in sorted(edges, key=lambda e: e[2]):
        if dsu.union(u, v):
            total += w
            mst_edges.append((u, v, w))
            accepted_weights.append(w)

    return total, mst_edges, accepted_weights

```

Figure 4 – Floyd-Warshall algorithm

Graph generation and benchmarking setup

Features:

- positive integer weights (default interval [1, 20])
- connected random baseline graphs for fair MST comparison
- multiple families for structural diversity
- repeated measurements (averaged)
- plotting: runtime curves, bar charts, heatmap

RESULTS

1. Experiment Scale and Correctness

Notebook outputs (main experiment):

- Scale rows: 55
- Family rows: 9
- Correctness check (same MST total): PASS

Example snapshot from the scale experiment (first few lines printed):

- $n=20$ (achieved density ≈ 0.10 due to forced connectivity) $\rightarrow$ Prim wins
- for slightly larger n at very low density, results already start to alternate

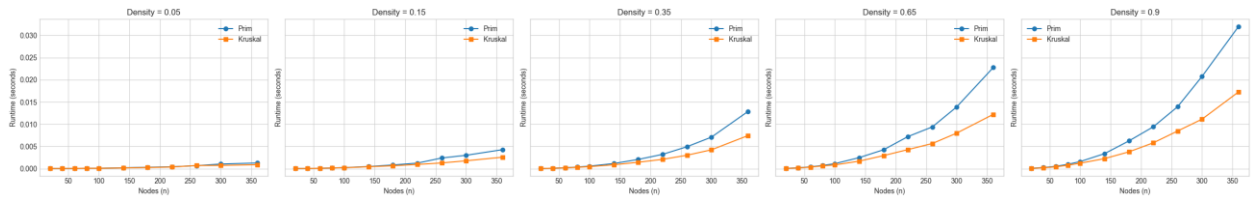


Figure 5. Results of runtime by number of nodes and density

2. Sparse vs Dense Behavior

Winner distribution by **target density** (55 cases total):

- density=0.05 → Prim=4, Kruskal=7
- density=0.15 → Prim=1, Kruskal=10
- density=0.35 → Prim=0, Kruskal=11
- density=0.65 → Prim=0, Kruskal=11
- density=0.90 → Prim=0, Kruskal=11

In this implementation and experiment design, **Kruskal dominates across almost all densities**, and the dominance becomes complete from density ≥ 0.35 .

3. Influence of Node Count

A robustness check was executed using larger graphs and median timing over 5 random graphs (each timed with 5 repeats). For $n = 200, 400, 600, 800$ and densities 0.05, 0.35, 0.90:

- Kruskal wins in all tested configurations
- The ratio K/P (Kruskal time / Prim time) ranged roughly from 0.388 to 0.797

Interpretation:

- Kruskal was consistently faster by about $\$1/0.797 \approx 1.25\times\$$ up to $\$1/0.388 \approx 2.58\times\$$ in these runs.

4. Graph Family Comparison

Family-level snapshot at $n \approx 120$ (density around 0.35 where applicable):

- Prim wins: cycle, tree
- Kruskal wins: simple, complete, wheel, bipartite, regular, grid, weighted

Observation:

- On extremely sparse families (tree, cycle, where $m \approx n$), Prim can be competitive or faster.
- For most other families, Kruskal's "single sort + DSU" pattern was faster in Python.

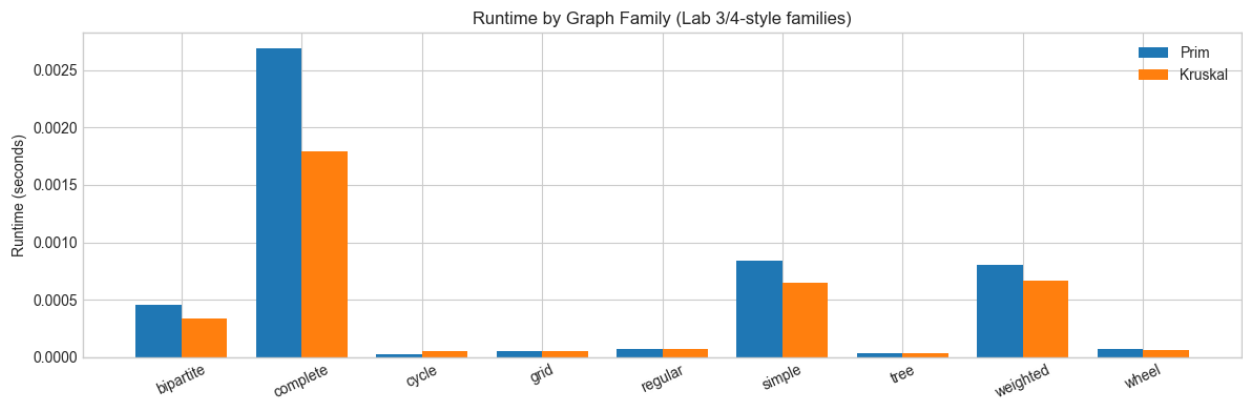


Figure 4. Results of runtime by the type of graph

5. Practical Rule Based on (n, m)

Based on the scale experiment (random connected graphs) and the robustness check:

Rule for this notebook's baseline implementations:

Use Kruskal as default.

Prim is only a good candidate on very sparse, near-tree structures.

A simple density-based version consistent with the observed results:

density ≥ 0.15 → Kruskal

otherwise → usually Kruskal, occasionally Prim on tiny graphs

Important implementation note (fairness check): The notebook also compared:

- the current “frontier-edge heap” Prim
- an optimized “key-based” Prim (decrease-key emulation)
- Kruskal

At $n=800$, density=0.35 (means over 5 trials):

- old Prim mean: 0.104918s
- key Prim mean: 0.017902s
- Kruskal mean : 0.044549s

So, with a better Prim implementation, Prim becomes the fastest (key-based Prim was ~5.86x faster than the old Prim and ~2.49x faster than Kruskal).

Discussion

1. Why Kruskal wins here:

Python's built-in sort is highly optimized and DSU operations are close to constant time

2. **Why baseline Prim loses on denser graphs:**

The frontier-edge heap approach may push many redundant edges and Heap operations dominate runtime

3. **Tradeoffs:**

Kruskal → strong default in Python for many graphs and Prim → becomes very strong with a key-based implementation

4. **Notes:**

Results depend on graph generator (connected-by-construction) and implementation details and Asymptotic complexity alone does not predict constant-factor behavior in Python

CONCLUSION

In the baseline implementations from the notebook, **Kruskal was faster in the majority of cases** and won all medium/high density buckets. **Prim can win on very sparse structured graphs** (tree/cycle) and becomes the best option when implemented in an optimized key-based form.

Final recommendation (practical):

- use Kruskal as the default MST algorithm in Python for random graphs and moderate/high density
- use Prim when the graph is extremely sparse or when you implement the key-based variant

GitHub repository

https://github.com/Nichita111/AA_labs